Unit - 4

* Principle of Backtracking.

→ backtracking is a systematic search technique for combinatorial problem.
→ It constructs a candidate incrementaly one decision at a time.
→ A state space tree represents the possible choises.
→ Depth - first search (DFS) is used to explore the tree.
→ If a partial solution cannot lead to valid soln, the branch is abandoned - this is pruning.
The process returns to the previous decision point & tries another choice.

Build → check → prune if valid → Backtrack → Try the next choice.

* Key Components.

- State space tree - all possible partial & complete solutions.
- Candidate generation - choose the next possible value / position.
- Bounding fn - tests whether the partial soln is promising
- Answer node - a complete solution satisfying all constraints
- Backtracking step - return to the previous level when no valid continuation exits.

Scanned with OKEN Scanner

* Control Abstraction.

```
Algorithm Backtrack (K).
// K = current level in the state space tree
    for each x[K] in T(x[1], ..., x[K-1]) do
        if bounding function (x[1], ..., x[K]) then
            if (x[1] ... x[K]) is an answer node
                write (x[1 : K]).
            else
                Backtrack (K+1)
            end if
        end if
    end for.
```

* Time Analysis of Control Abstraction.

Let m = number of choices available at each level.

Let n = maximum depth of state space tree.

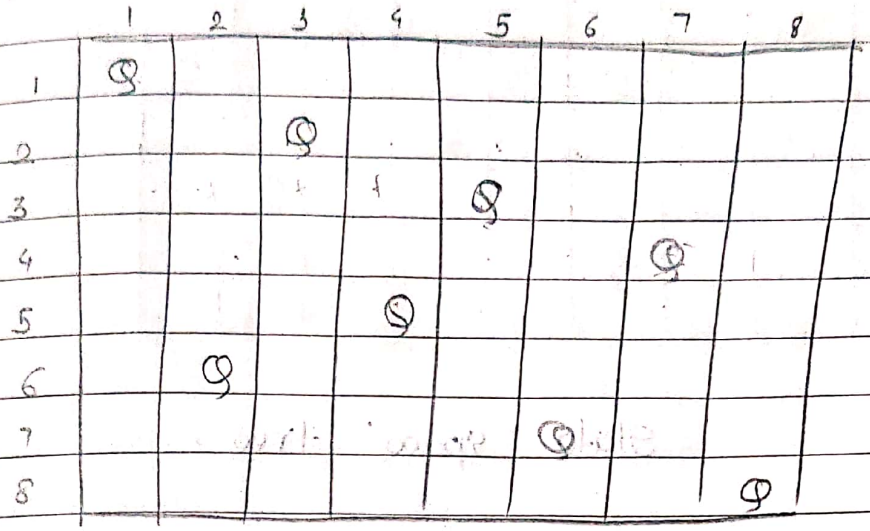
Let p = time required to evaluate the bounding function.

In the worst case, the algo may visit a large portion of state space tree. An illustrative upper bound form is $O(m^n \times p)$.

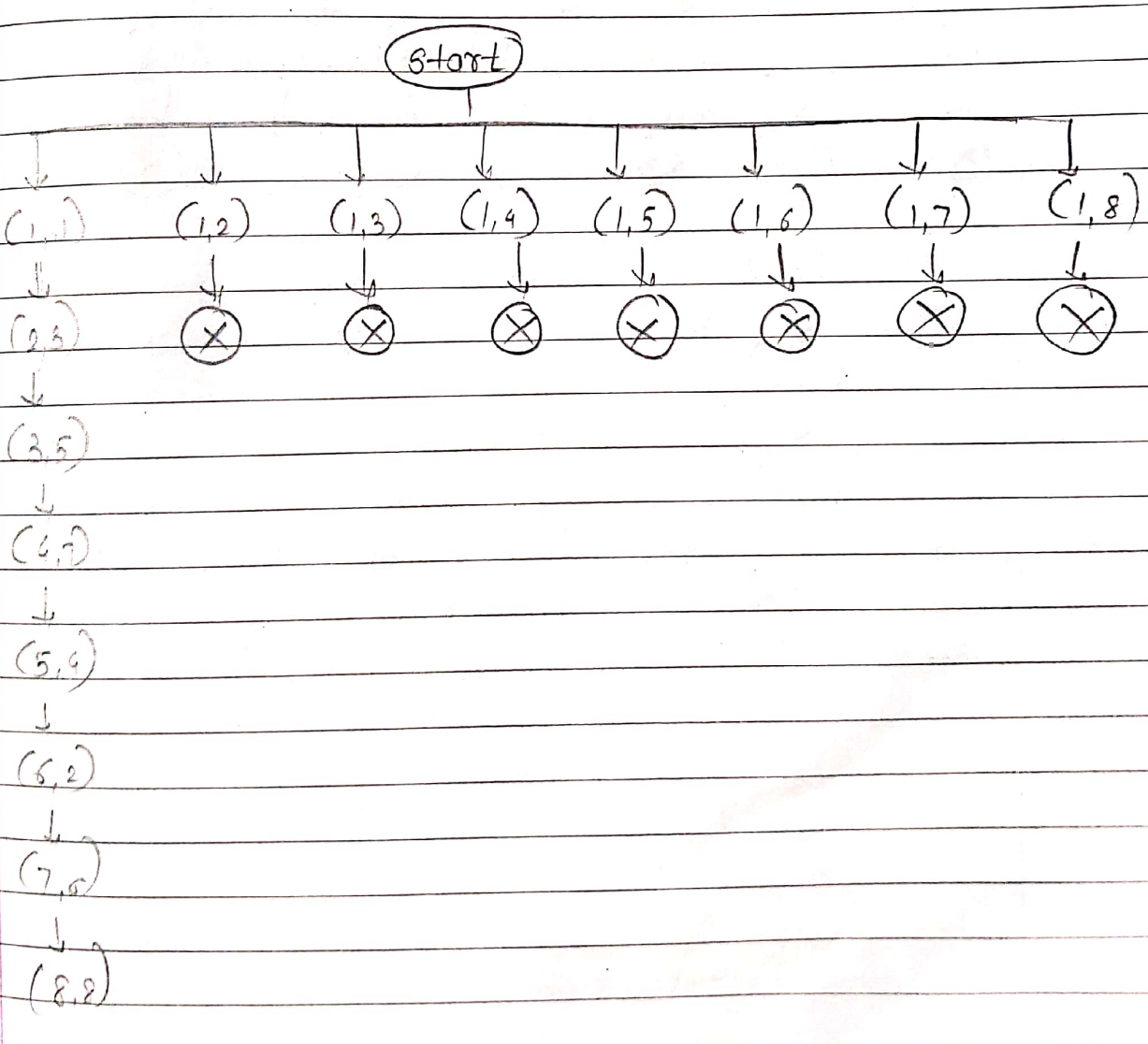
Therefore, backtracking generally has exponential worst case running time.

Effective pruning can make practical execution much faster than brute force enumeration.

1. State space tree. (8 Queen problem)



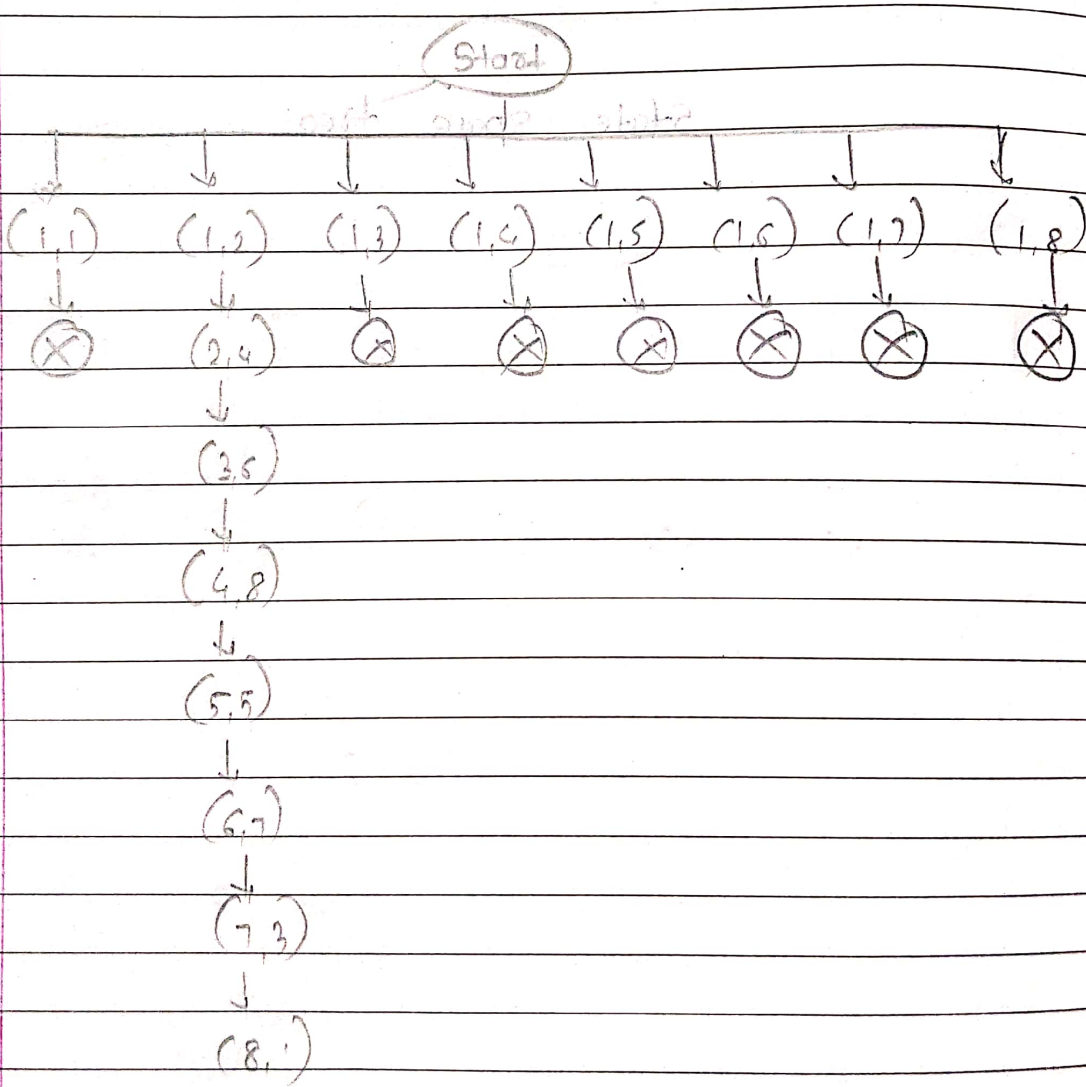
State space tree.



(2)

	1	2	3	4	5	6	7	8
1		Q						
2				Q				
3						Q		
4								Q
5					Q			
6							Q	
7			Q					
8	Q							

State space tree



Branch and Bound.

Principle of branch and bound.

The B & B technique maintains a state space tree.

1] Start with root node representing the Complete problem.
2] Branch the root into smaller subproblems.
3] Calculate a bound for every generated node.
4] Compare the bound with the current best Solution (incumbent).
5] If a node cannot produces a better solution prune that node.
6] otherwise expand the node further.
7] Continue until no promising lives nodes remain

Control Abstraction.

```
Branch and Bound ()
1] construct the root node
2] calculate the bound of the root.
3] insert the root into the list of live nodes
4] while live nodes are available do
        select live node u
        if u is not promising
            discard u
        else
            Generate children of u
            for each child v do
                calculate Bound (v)
                if
```

if v is promising
    insert v into live nodes.
else
    prune v

5) Return the best solution found.

There are 3 approaches in order to implement branch & bound.

FIFo → queue → breadth-oriented search (BFS)
LIFo → stack → depth-oriented search (DFS)
LC → priority queue → expand the live node with most promising bound (typically smallest lower bound for minimization).

Travelling Salesman problem

Ex-

	1	2	3	4	5	
1	∞	20	30	10	11	10
2	15	∞	16	4	2	2
3	3	5	∞	2	4	2
4	19	6	18	∞	3	3
5	16	4	7	17	∞	4

min rows

	1	2	3	4	5
1	∞	10	20	0	1
2	13	∞	14	2	0
3	1	3	∞	0	2
4	16	3	15	∞	0
5	12	0	3	12	∞
	1	0	3	0	0

→ min. Columns

(1)

	1	2	3	4	5
1	∞	10	17	0	1
2	12	∞	11	2	0
3	0	3	∞	∞	2
4	15	3	12	∞	0
5	11	0	0	12	∞

reduce minimum values from rows & column.

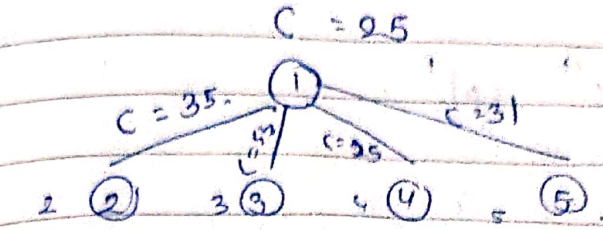
(2) $10+2+2+3+4+1+3 = 25$. → add cost of rows & col^m.

→

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	0
2	∞	∞	11	2	0	0
3	0	∞	∞	0	2	0
4	15	∞	12	∞	0	0
5	11	∞	0	12	∞	0
	0	0	0	0	0	

No cost reduction.

Scanned with OKEN Scanner



1st row → 2nd col^
1st row → 3rd col^
1st row → 4
1st row → 5

$$c[1,2] = c[1,2] + \text{Parent}(\gamma) + \gamma^{\wedge}$$
$$= 10 + 25 + 0$$
$$= 35$$

(3) 1st row → 3rd col^.

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	12	∞	∞	11	0	0
3	∞	3	∞	∞	2	0
4	15	3	∞	12	0	0
5	11	0	∞	0	∞	0
	11	0		0	0	

do min col^ reduce

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	1	∞	∞	11	0	
3	∞	3	∞	0	2	
4	4	3	∞	12	0	
5	0	0	∞	0	∞	
	0	0		0	0	

$$c[1,3] = c[1,3] + \gamma + \gamma^{\wedge}$$ → reduction if any.
$$= 17 + 25 + 11$$
$$= 53.$$

④. 1st row → 4th col^{n}.

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	12	∞	11	∞	0	0
3	0	3	∞	∞	2	0
4	15	3	12	∞	0	0
5	11	0	0	∞	∞	0
	0	0	0		0	

$$c[1,4] = c[1,4] + 2 + 8^{n}$$
$$= 0 + 25 + 0$$
$$= 25.$$

⑤ 1st row → 5th col^{n}.

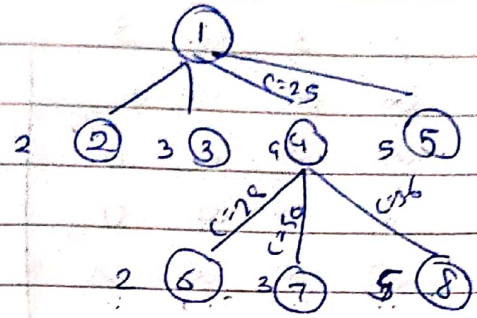
	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	12	∞	11	2	∞	2
3	0	3	∞	0	∞	0
4	15	3	12	∞	∞	3
5	∞	0	0	12	∞	0

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	10	∞	9	0	∞
3	0	3	∞	0	∞
4	12	0	4	∞	∞
5	∞	0	0	12	∞
	0	0	0	0	

Scanned with OKEN Scanner

$$c[1,5] = c[1,5] + \gamma + \gamma''$$
$$= 1 + 25 + 2 13$$
$$= 31.$$

Now,



(4) now parent is (4).

(4)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	11	∞	0
3	0	3	∞	∞	2
4	15	13	12	∞	0
5	11	0	0	∞	∞

4th row 2nd Colm → ∞.

4 → 4 is ~~not~~ ∞ 2 → 1 is ∞.

(6)

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	∞	∞	11	∞	0	0
3	0	∞	∞	∞	2	0
4	∞	∞	∞	∞	∞	
5	11	∞	0	∞	∞	0
	0		0		0	

$$c[4,2] = 3 + 25 + 0 = 28.$$

Scanned with OKEN Scanner

(7)

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	12	∞	∞	∞	0	0
3	∞	3	∞	∞	2	2
4	∞	∞	∞	∞	∞	
5	11	0	∞	∞	∞	0

4th row
3rd col

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	∞	∞	0
3	∞	1	∞	∞	0
4	∞	∞	∞	∞	∞
5	11	0	∞	∞	∞
	11	0			0

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	∞	∞	0
3	∞	0	∞	∞	∞
4	∞	∞	∞	∞	∞
5	0	0	∞	∞	∞

$e[4,3] = 12 + 25 + 2 + 11 + 0$ (14, 12, 26)

$= 26 + 25$

$= 50$

(8)

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	12	∞	11	∞	∞	11
3	0	3	∞	∞	∞	0
4	∞	∞	∞	∞	∞	
5	∞	0	0	∞	∞	0

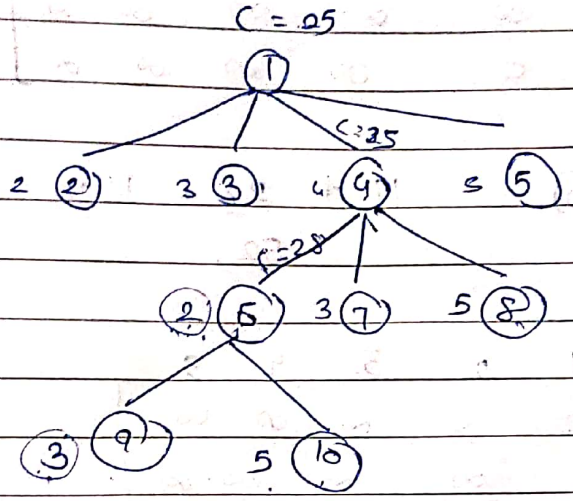
Scanned with OKEN Scanner

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	1	∞	0	∞	∞
3	0	3	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	0	0	∞	∞

0 0 0

$C[4,5] = 0 + 25 + 11$

$= 36.$



r^{th} point.

(6)

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	11	∞	0
3	0	∞	∞	∞	2
4	∞	∞	∞	∞	∞
5	11	∞	0	∞	∞

~~2~~ th row → 3rd col^{m} 3 → 1 ∞.

(9)

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	∞	∞	∞	∞	∞	
3	∞	∞	∞	∞	2	2
4	∞	∞	∞	∞	∞	
5	11	∞	∞	∞	∞	11

1	∞	∞	∞	∞	∞
2	∞	∞	∞	∞	∞
3	∞	∞	∞	∞	0
4	∞	∞	∞	∞	∞
5	0	∞	∞	∞	∞

$c[2,3] = 17 + 28 + 11+2$

$= 52.$

(10)

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	
2	∞	∞	∞	∞	∞	
3	0	∞	∞	∞	∞	0
4	∞	∞	∞	∞	∞	
5	∞	∞	0	∞	∞	0

$c[2,5] = 0 + 28 + 0$

$= 28$

now find

1st parent

(10) 5

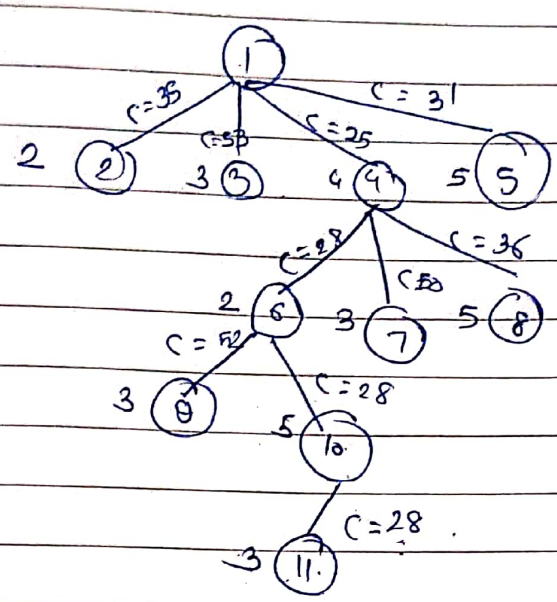
(11) 5

5th row → 3rd col.

N]

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	∞	∞	∞
3	∞	∞	∞	∞	∞
4	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞

c[5,3]
= 0 + 28 + 0



(path) Flow → 1 → 4 → 2 → 5 → 3 → 1

final cost → 28.

Flow → 1 → 4 → 2 → 5 → 3 → 1

Scanned with OKEN Scanner